IoT-Based Smart Obstacle Avoiding Robot Using Arduino and ESP32 with Real-Time Cloud Telemetry

R Dakshan Ganesh S Shyam Kumar R Arun Adithya Ms. Sowmiya

Department of Artificial Intelligence & Machine Learning, Vels Institute of Science, Technology & Advanced Studies (VISTAS), Pallavaram, Chennai – 600117, India

***Abstract*—Small-scale autonomous robots are often either purely reactive and invisible to their operators, or built on costlier single-board computers. This paper presents an IoT-based smart obstacle avoiding robot that combines reliable local avoidance with real-time cloud telemetry on a student-level budget. An Arduino Uno performs the time-critical sense–decide–act loop using an HC-SR04 ultrasonic sensor on an SG90 scanning servo and an L293D motor shield driving four TT geared motors. An ESP32 acts as a dedicated Wi-Fi bridge, receiving one-way state lines from the Uno through a 5 V-to-3.3 V resistor divider and publishing front, left and right distances and the robot state to a ThingSpeak channel at 20-second intervals. The full hardware build, firmware and serial bridge have been completed, and the ESP32 monitoring dashboard has been verified in both Auto and Manual modes, including Emergency Stop. Bench testing is complete and floor testing with threshold tuning is in progress; quantitative floor-test results are therefore not yet reported. The design is intended for indoor educational, warehouse-aisle and inspection prototypes.**

***Index Terms*—Arduino Uno, ESP32, Internet of Things (IoT), obstacle avoiding robot, HC-SR04 ultrasonic sensor, L293D motor driver, embedded systems, real-time monitoring, ThingSpeak.**

I. INTRODUCTION

Autonomous mobile robots are increasingly used for indoor inspection, warehouse logistics and education. Practical deployments, however, face three recurring obstacles: hardware cost, technical complexity, and the lack of real-time remote oversight in small-scale systems. Low-cost Arduino-only robots typically run a reactive stop-and-turn loop and give the operator no visibility once the robot is moving on the floor, so sensor faults, stuck runs and badly tuned thresholds can only be diagnosed by watching the robot in person.

This work addresses that gap with a dual-controller architecture. An Arduino Uno keeps the fast, deterministic control loop local, while an ESP32 is used purely as a Wi-Fi telemetry bridge to the cloud. The guiding question is how a student-budget robot can avoid obstacles reliably and also report what it is doing, in real time, from anywhere with Wi-Fi.

The main contributions are: (i) a complete low-cost sense → decide → act → report pipeline built from off-the-shelf modules; (ii) a documented 5 V-to-3.3 V serial bridge between the Uno and the ESP32; (iii) a cloud dashboard that exposes front, left and right distances and the robot state; and (iv) a staged test plan covering bench and floor operation.

II. RELATED WORK

Borenstein and Koren introduced the Vector Field Histogram (VFH) method in 1991, which established the classical foundation for fast local reactive steering [1]. Mothe *et al.* reported an IoT-based obstacle avoidance robot, which supports combining cloud telemetry with basic avoidance [2]. Ekta *et al.* confirmed the suitability of ultrasonic sensors for indoor navigation [3], and Aishwarya *et al.* describe an Arduino obstacle avoiding robot that provides a baseline Uno + HC-SR04 architecture [5]. A review of path planning methods by Katona *et al.* [4] supports the use of simple threshold logic rather than complex heuristics at this hardware scale. Finally, ESP32-based remote telemetry bridges described in recent IoT robotics surveys support the dual-controller pattern adopted here.

A. Research Gap

Classical local-navigation methods and most Arduino-only robots operate entirely offline. IoT-enabled platforms in the literature typically rely on costlier single-board computers. Few student-scale projects validate a complete sense–decide–act–telemeter pipeline against a live cloud dashboard, and reliable low-voltage bridging between a 5 V Arduino and a 3.3 V ESP32 is rarely documented in detail for beginner builds. The present work targets this combination: a low-cost Uno + ESP32 architecture with a real-time cloud dashboard for a reactive obstacle avoiding robot.

III. PROBLEM STATEMENT AND OBJECTIVES

Small-scale robots are either purely reactive or expensive, and low-cost microcontroller builds provide no remote visibility. Operators cannot verify sensor health, tune safe-distance thresholds or diagnose failed runs without physically observing the robot. The objectives of this work are:

1) **Sense:** detect obstacles with an HC-SR04 mounted on a scanning servo.
2) **Decide and act:** run stop / reverse / scan / turn logic locally on the Arduino Uno.
3) **Bridge:** forward telemetry from the Uno to the ESP32 over a level-shifted serial link.
4) **Monitor:** stream live distance and state data to a ThingSpeak dashboard.
5) **Validate:** test on the bench and the floor for safe, repeatable avoidance.

6) **Optimise cost:** keep the build affordable using off-the-shelf modules.

IV. PROPOSED SYSTEM

A. Architecture

Fig. 1 shows the system architecture, organised into a power supply, a local processing and control section, and a remote gateway and cloud section. The Arduino Uno handles ultrasonic ranging, servo scanning and four-motor driving. The ESP32 receives state lines from the Uno through the serial bridge and uploads them to the cloud over Wi-Fi. Splitting the work this way keeps real-time control deterministic while still allowing remote observation.

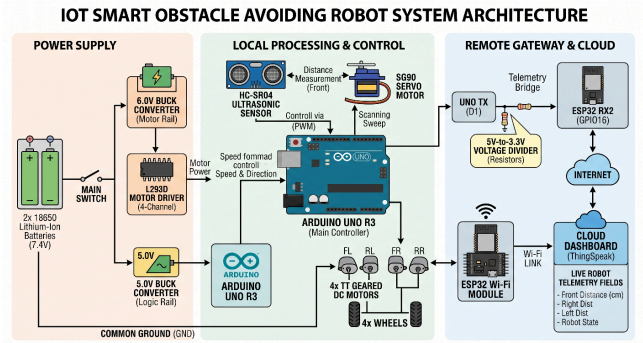


Fig. 1. System architecture of the IoT smart obstacle avoiding robot.

B. Hardware

The robot uses a four-wheel-drive chassis with four TT geared DC motors (3–6 V). Table I lists the main components. The Adafruit Motor Shield V1 (L293D) drives the motors through the AFMotor library, so the firmware is not compatible with Motor Shield V2 without changes.

TABLE I. MAIN COMPONENTS

Component	Role
Arduino Uno R3	Local sense–decide–act controller
ESP32 DevKit	Wi-Fi telemetry bridge to the cloud
HC-SR04	Ultrasonic front-distance sensing
SG90 servo	Left/right scanning sweep
L293D motor shield	Four-channel DC motor driving
4 × TT motors, 4WD chassis	Locomotion
2 × 18650 cells (2S)	7.4 V nominal (8.4 V full) source
6 V and 5 V buck converters	Motor rail and logic rail

C. Power Design

The motors and the controllers are powered from separate regulated rails derived from the 2S 18650 pack. A buck converter set to 6.0 V feeds the motor driver, and a second converter set to 5.0 V (rated at least 3 A) feeds the Uno and the ESP32. A 470–1000 µF capacitor is placed across the motor supply near the driver to reduce brownouts, and all modules share a common ground. This separation protects the logic boards from motor noise and prevents the controller resets that occur when motors are run from the Uno 5 V pin.

D. Serial Bridge

The Uno TX pin (D1) is connected to ESP32 RX2 (GPIO16) through a 1 kΩ / 2 kΩ resistor divider, which reduces the 5 V serial level to approximately 3.3 V. The link is one-way, the ESP32 TX pin is left unconnected, and both boards share ground. Connecting a 5 V Uno TX output directly to an ESP32 input is not permitted.

V. METHODOLOGY

A. Runtime Control Loop

The Uno repeats a four-stage loop of sensing, deciding, acting and reporting:

7) **Sense:** ping the HC-SR04 for the front distance every cycle (NewPing, 200 cm maximum range; no echo is treated as clear).
8) **Decide:** compare the distance with the 30 cm safe threshold. If the path is clear the robot continues forward.
9) **Act:** on an obstacle, stop, reverse for 350 ms, sweep the servo to 25° and 155° to measure right and left clearance, then turn toward the clearer side for about 480 ms.
10) **Report:** emit a telemetry line over serial after each state change.

Left and right distances are compared with a simple rule: if the left distance is greater than or equal to the right distance the robot turns left, otherwise it turns right. This threshold logic is deliberately simple, in line with the finding that complex heuristics are unnecessary at this scale [4].

B. Telemetry Format

Each cycle the Uno prints a single line, for example *distance=24;left=85;right=41;state=turning*left_. The ESP32 parses the key–value pairs and writes them to four ThingSpeak fields. Because the free ThingSpeak channel limits write frequency, uploads are throttled to one every 20 s while the local control loop runs at full speed. Table II lists the fields.

TABLE II. TELEMETRY FIELDS

Field	Description
distance	Front distance in cm (HC-SR04)
left / right	Scanned side distances in cm
state	ready, moving*forward, obstacle*detected, turning*left, turning*right, turn_complete

C. Development Process

Development followed five stages: component procurement, chassis assembly and wiring, programming of the Uno and ESP32, integration of the cloud dashboard, and testing and tuning. The robot has no external training dataset; its working data is the live telemetry stream, which is used to validate behaviour and tune the safe-distance threshold.

VI. IMPLEMENTATION

The implementation is divided into six modules. The sensing module uses NewPing on the Uno to return the front distance each cycle. The servo scan module sweeps the SG90 to 25° and 155° to sample side clearance when an obstacle is detected. The motor control module uses AFMotor to drive the four TT motors for forward, reverse, left-turn and right-turn motion, with the left motors on M1/M2 and the right motors

on M3/M4 and a default speed of 170 out of 255. The serial bridge module carries one-way state lines to the ESP32. The cloud telemetry module parses these lines and posts the fields to ThingSpeak over Wi-Fi. Finally, the dashboard module presents the mode, distances and manual/auto controls to the operator.

The Uno firmware is built with the Arduino IDE using the AFMotor, NewPing and Servo libraries. The ESP32 firmware uses the Wi-Fi and HTTP client libraries and is uploaded with the ESP32 board package. During development the Uno code is uploaded first, with the serial wire disconnected if uploads fail, and its serial monitor (9600 baud) is used to confirm the ready state before the ESP32 (115200 baud) is programmed.

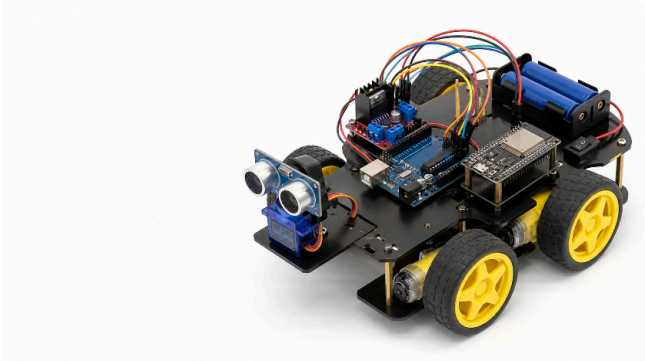


Fig. 2. Assembled four-wheel robot with Uno, ESP32, motor shield and scanning sensor head.

VII. TESTING AND RESULTS

A. Test Procedure

Testing follows two stages. In the bench test the chassis is lifted so the wheels cannot move; the logic rail is powered first, then the motor rail, and an obstacle is placed 20–30 cm in front of the sensor to confirm that the robot stops, reverses and turns, and that the ESP32 and cloud channel show updated values. In the floor test the robot runs on a flat, dry floor in a wide clear area using the default 30 cm safe distance, and the motor speed and the 480 ms turn delay are tuned until turns are reliable without over-rotating.

B. Current Progress

At the time of writing the following have been achieved: the hardware is fully assembled (chassis, motors, sensor and servo head, dual power rails); the Uno and ESP32 firmware are uploaded and the serial bridge is verified with live telemetry lines; and the ESP32 dashboard is confirmed working in Auto and Manual modes, including Emergency Stop. Bench testing is complete, while floor testing and threshold tuning are in progress.

C. Live Obstacle Test

Fig. 3 shows the dashboard during an obstacle event. The front, left and right distances all read 4 cm, the obstacle flag is DETECTED and the mode is AUTO, which is consistent with the 30 cm safe-distance trigger in the firmware. Fig. 3(b) shows the Manual mode controls and the Emergency Stop button.

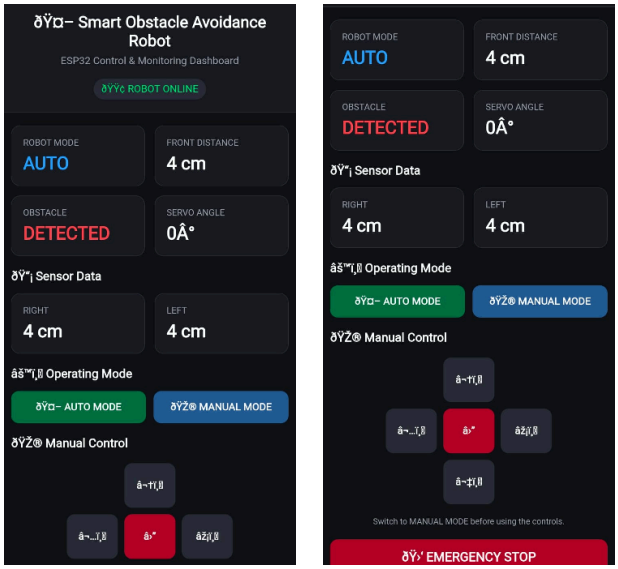


Fig. 3. Dashboard: (a) Auto mode with live readings; (b) Manual mode and Emergency Stop.

TABLE III. STATUS OF VERIFICATION

Item	Status
Hardware assembly and dual power rails	Complete
Uno and ESP32 firmware, serial bridge	Verified
Dashboard: Auto, Manual, Emergency Stop	Verified
Bench testing	Complete
Floor testing, threshold and motor tuning	In progress

No quantitative performance figures, such as avoidance success rate over repeated floor runs, are reported here, because floor testing and tuning are still under way. These measurements are planned for the final evaluation.

VIII. APPLICATIONS AND LIMITATIONS

Intended uses include indoor inspection demonstrations, warehouse-aisle training prototypes, educational laboratory models for sensors, motors and IoT telemetry, and campus logistics concepts. The current design has clear limitations:

- Avoidance is reactive only; the robot builds no map and plans no global route.
- HC-SR04 accuracy depends on object shape, material and angle, and a single sensor struggles with soft, narrow or sharply angled objects.
- Remote monitoring depends on Wi-Fi availability, and the free ThingSpeak channel limits updates to about one every 20 s.
- The system is suitable for lightweight indoor prototypes and is not a safety-critical industrial vehicle.

IX. FUTURE ENHANCEMENTS

Planned improvements are wheel encoders for speed and distance feedback; MQTT telemetry for faster and more scalable messaging; a camera or LiDAR for richer sensing and

mapping; battery-voltage monitoring published to the dashboard; remote manual commands from the cloud with a fail-safe emergency stop; and a single-controller redesign using one ESP32 as the primary controller.

X. CONCLUSION

This paper presented a low-cost IoT-based obstacle avoiding robot in which an Arduino Uno provides deterministic local avoidance and an ESP32 provides real-time cloud visibility. The hardware build, firmware, level-shifted serial bridge and dashboard have been completed and verified, and the project has moved into floor testing and threshold tuning. The results so far indicate that the core sense → decide → act → report loop works as designed. Final floor-test measurements will be needed to quantify avoidance reliability.

ACKNOWLEDGMENT

The authors thank their project guide, Ms. Sowmiya, Department of Artificial Intelligence & Machine Learning, VISTAS, for her guidance.

REFERENCES

[1] J. Borenstein and Y. Koren, “The vector field histogram—fast obstacle avoidance for mobile robots,” *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 278–288, Jun. 1991.

[2] R. Mothe *et al.*, “IoT based obstacle avoidance robot,” 2020.

[3] K. Ekta *et al.*, “Obstacle avoiding robot,” 2021.

[4] K. Katona *et al.*, “Path planning methods—A review,” 2024.

[5] V. Aishwarya *et al.*, “Arduino obstacle avoiding robot,” 2024.

[6] Arduino, “Arduino documentation.” [Online]. Available: https://docs.arduino.cc/

[7] Espressif Systems, “Arduino-ESP32 core documentation.” [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/

[8] Adafruit Industries, “Adafruit Motor Shield (V1) guide.” [Online]. Available: https://learn.adafruit.com/adafruit-motor-shield/

[9] MathWorks, “ThingSpeak documentation.” [Online]. Available: https://www.mathworks.com/help/thingspeak/